

## Module 11

Topics: Intro to macros

*Files to download:*

### 11.1 – Intro to Macros

SAS macro code has 2 parts: macros and macro variables. Macro names begin with a % and macro variables begin with a &.

Let's make a quick dataset to demonstrate some of the things you can do with macros and macro variables. The dataset mac\_v1 has 3 variables: studyid and 2 other numeric fields called var1 and var2.

```
data mac_v1;
  input studyid var1 var2;
  cards;
1001 4 1
1002 2 7
1003 8 1
1004 0 4
1005 8 9
;
```

First let's take a look at a macro variable. Macro variables are assigned using the LET statement.

If we run the following:

```
%let newvar = var3;
```

And then use that macro variable in the following code:

```
data mac_v2;
  set mac_v1;
  &newvar = var1 + var2;
run;
```

SAS basically does a find and replace to convert the data step above to this:

```
data mac_v2;
  set mac_v1;
  var3 = var1 + var2;
run;
```

This particular example obviously doesn't save us much time or typing, but you can see how this might be handy. I could write an entire program, hundreds of lines of code, referencing the macro variable in my LET statement, and by simply changing the name of the variable in the LET statement once at the top of my program I can execute that entire program again with a different variable.

Another thing we might want to do is use a macro to transform a variable into several new ones. Macros always begin with **%macro** and end with **%mend**. Anything placed between the **%macro** and **%mend** will be processed when the macro is run. A period is used to indicate that the value of a macro variable should be connected to the next item.

The following code creates a macro called `transform_this` which will create 2 new variables, one to sum some variables and one to get the difference between them. When we run this code it doesn't actually do anything except create the macro `transform_this` which can then be called later.

```
%macro transform_this(x);

    &x._sum = var1 + var2;
    &x._difference = var1 - var2;

%mend transform_this;
```

Because the sum and difference code in this macro would only work inside of a data step, that is how we will need to call the macro to run on a specific dataset. In the following code we will run this macro on the original `mac_v1` dataset. The (x) has been replaced with the names I want to use for my new variables – notice how SAS will find and replace the text “x” from the previous step with “var3” or “var4”. Note that the &x will be replaced with the value of the item in the parenthesis and the period indicates that this should be immediately followed by the rest of the string, in this case “\_sum” or “\_difference”.

```
data mac_v3;
    set mac_v1;
        %transform_this(var3);
        %transform_this(var4);
run;
```

| Obs | studyid | var1 | var2 | var3_sum | var3_difference | var4_sum | var4_difference |
|-----|---------|------|------|----------|-----------------|----------|-----------------|
| 1   | 1001    | 4    | 1    | 5        | 3               | 5        | 3               |
| 2   | 1002    | 2    | 7    | 9        | -5              | 9        | -5              |
| 3   | 1003    | 8    | 1    | 9        | 7               | 9        | 7               |
| 4   | 1004    | 0    | 4    | 4        | -4              | 4        | -4              |
| 5   | 1005    | 8    | 9    | 17       | -1              | 17       | -1              |

Because of our macro, SAS basically created and ran the following code. Note how SAS replaced the “&x.” with the text from the variable names we provided.

```
data mac_v3;
    set mac_v1;
        var3_sum = var1 + var2;
        var3_difference = var1 - var2;
        var4_sum = var1 + var2;
```

```
var4_difference = var1 - var2;
run;
```

Now that you are getting the hang of how this works, a slightly more useful example might be to do various calculations on several variables. The macro `transform_that` will take a variable of our choice and create one new variable to square the value, one to cube the value, and one to get the inverse of the value.

```
%macro transform_that(x);
    &x._squared = &x ** 2;
    &x._cubed = &x ** 3;
    &x._inverse = 1 / &x;
%mend transform_that;
```

And to run it we use the following code. Note how SAS created 3 new variables with the calculations.

```
data mac_v4; set mac_v1;
    %transform_that(var1);
run;
```

| Obs | studyid | var1 | var2 | var1_squared | var1_cubed | var1_inverse |
|-----|---------|------|------|--------------|------------|--------------|
| 1   | 1001    | 4    | 1    | 16           | 64         | 0.250        |
| 2   | 1002    | 2    | 7    | 4            | 8          | 0.500        |
| 3   | 1003    | 8    | 1    | 64           | 512        | 0.125        |
| 4   | 1004    | 0    | 4    | 0            | 0          | .            |
| 5   | 1005    | 8    | 9    | 64           | 512        | 0.125        |

Here is what it would look like if we wanted to do this for more than one variable at a time.

```
data mac_v5; set mac_v1;
    %transform_that(var1);
    %transform_that(var2);
run;
```

| Obs | studyid | var1 | var2 | var1_squared | var1_cubed | var1_inverse | var2_squared | var2_cubed | var2_inverse |
|-----|---------|------|------|--------------|------------|--------------|--------------|------------|--------------|
| 1   | 1001    | 4    | 1    | 16           | 64         | 0.250        | 1            | 1          | 1.00000      |
| 2   | 1002    | 2    | 7    | 4            | 8          | 0.500        | 49           | 343        | 0.14286      |
| 3   | 1003    | 8    | 1    | 64           | 512        | 0.125        | 1            | 1          | 1.00000      |
| 4   | 1004    | 0    | 4    | 0            | 0          | .            | 16           | 64         | 0.25000      |
| 5   | 1005    | 8    | 9    | 64           | 512        | 0.125        | 81           | 729        | 0.11111      |

You can start to see how you can use macros to process repetitive code on many variables with one small block of code.

Macros can be used for anything you will be doing over and over again and would prefer not to type out lots of the same syntax when all that is changing is a variable or dataset name.

As another example, imagine you have been asked to run PROC CONTENTS on several datasets. Rather than typing *PROC CONTENTS DATA = DATASETNAME; RUN;* several times, you could create a macro to simplify. The macro `contents_of` will process the proc contents code for any dataset listed as the DSN.

```
%macro contents_of(DSN);
    proc contents data=&DSN;
    run;
%mend contents_of;

%contents_of(mac_v1);
%contents_of(mac_v2);
%contents_of(mac_v3);
%contents_of(mac_v4);
%contents_of(mac_v5);
```

You could do the same with PROC PRINT.

```
%macro print_it(DSN);
    proc print data = &DSN;
    run;
%mend print_it;

%print_it(mac_v1);
%print_it(mac_v2);
%print_it(mac_v3);
%print_it(mac_v4);
%print_it(mac_v5);
```

Imagine how helpful this would be if you had long DATA or PROC steps with many options. You could type them out only once in the macro and process them multiple times on several different datasets. A few other handy things you can do with macros and macro variables. Note that you can't run any of the code below, this is just FYI.

- In Module 6, we saw how global macro variables could be used to stamp a system date on a file in a format of our choice:
 

```
data _NULL_;
    call symput ("dt", put(today(), mmddyy.));
run;
```
- %LET can be used to reference an entire pathway:
 

```
%let output_folder = H:\AEPI537D;
```
- %INCLUDE can be used to run an entire SAS program:
 

```
%include 'H:\AEPI537D\SAS Code\Mod10.sas';
```
- You can use macros and macro variables to create all kinds of shortcuts. In the following example I am processing the same series of steps on 3 different longitudinal datasets that contain the same variables over time. I can use macros to process the same series of steps for several different datasets. I'm using macros to manipulate the dataset being used, the pathway being saved to, the suffix on datasets being created, etc. The possibilities are endless!

```
libname month6 'H:\AEPI537D\StudyData\Month6';
libname month12 'H:\AEPI537D\StudyData\Month12';
libname month18 'H:\AEPI537D\StudyData\Month18';
```

```
%macro lots_of_stuff (data_in = , libname_out = , visit_suffix = );  
    /* tons and tons of data cleaning and processing steps in here */  
    /* you can even create more datasets in here using macro vars */  
    data &libname_out..surveystuff_&visit_suffix.;  
        set temp1;  
    run;  
  
    data &libname_out..eventstuff_&visit_suffix.;  
        set temp2;  
    run;  
  
%mend lots_of_stuff;  
  
%lots_of_stuff (data_in = full_month6, libname_out = month6, visit_suffix = m6);  
%lots_of_stuff (data_in = full_month12, libname_out = month12, visit_suffix = m12);  
%lots_of_stuff (data_in = full_month18, libname_out = month18, visit_suffix = m18);
```

**MOD 11: USE SAS COMMENTS TO NUMBER YOUR WORK.**

Using the pre-loaded SAS dataset in the SASHELP library called CLASS (SET SASHELP.CLASS):

1. Create a macro (name is up to you) that will generate a PROC PRINT and a PROC CONTENTS, and use a macro variable for the dataset name so that it could be run on multiple datasets. For this example you can run it just once on the SASHELP.CLASS dataset.
2. Create a macro (name is up to you) that will add 10 to the value of a numeric variable and store it in a new field. Try running this macro on the Age variable in the SASHELP.CLASS dataset.